

First important step

As the size of the font contained inside terminals is quite small by standard, the professor provided us with a script to increase it. It must be run every time we restart the Kathara virtual machine.

Here is the code for `resize_xterm.sh` :

```
xrdb -merge .Xresources
```

It can be run with:

```
./resize_xterm.sh
```

Configuring the environment

Obviously, before starting the lab, we need to configure the parameters of the lab. This can be done in the `lab.conf` file, which can be created inside the directory for the laboratory.

Inside this file, we can configure interfaces and the respective network they're connected to.

This is an example:

```
r1[0]='A'
r1[bridged]=true
r1[sysctl]="net.ipv6.conf.all.forwarding=1"
r1[sysctl]="net.ipv4.ip_forward=1"

pc1[0]="A"
pc2[0]="A"
pc3[0]="A"
```

Basically, we're saying that the network to which the `eth0` interface of device `r1` is `A`. This also applies to `pc1`, `pc2` and `pc3`.

For `r1`, we're also configuring some extra parameters, like allowing the forwarding of packets with the IPv6 protocol, and the same with IPv4. This is done through the `sysctl` command.

Also, remember to allow through `kathara settings` some options. Specifically:

- enable IPv6 through option 9.
- modify the default network driver through option 10, selecting Kathara with Linux bridges.

Finally, we need to create the `device.startup` file in the environment. Device could be `pc1` , but also `r1` , and so on.

Assigning IP addresses to interfaces

Once the lab file has been configured, we need to assign IP addresses to the interfaces we've defined in `lab.conf` .

Addresses can be assigned in different ways. We can distinguish between:

- static addressing
- dynamic addressing

For static addressing, we can assign addresses in three main ways:

- Through the interfaces file.
- Using `ip` commands.
- Using `ifconfig` command

Let's analyze the first method. For it, we need to create the directory for the device, like `pc1/etc/network` . Inside it, we place the `interfaces` file. It could be something like this:

```
auto eth0
iface eth0 inet static
    address 192.168.100.25/29
    gateway 192.168.100.30
    dns-nameservers 8.8.8.8
```

With `address` , we're specifying the address we want to assign to the interface. The `gateway` , instead, is used to route all packets that do not match any routing table entries to that address. The `dns-nameservers` is used to define DNS servers.

Once we've defined the interface in this way, we need to make it work by adding this command in the startup file:

```
ip addr flush eth0
ifup eth0
```

The second method is given by manual configuration through the `ip` commands. This can be done inside the startup files, like in this way:

```
ip addr flush eth0
ip addr add 192.168.100.26/29 dev eth0
```

The third is given by the use of the `ifconfig` commands, which can be inserted inside the startup file as well.

```
ip addr flush eth0
ifconfig eth0 192.168.100.27 netmask 255.255.255.248 up
```

Once we've done this, we've defined the correct static addresses.

Configuring static routes

We've seen with `pc1` how to configure, in the interfaces file, the default gateway. To allow communication through devices which are not directly connected, we need to define some rules of routing. This especially for routers, who's task is to take packets and redirect them to the correct interface depending on the rules.

This concept applies, as well, to default gateways. If we want to send a packet outside the network, we have to send it to the default gateway router, who's task is to communicate with the rest of the internet.

For `pc2`, we need to add an option to allow the routing to the default gateway. This can be done with the `ip` command, inside the `pc2.startup`.

```
ip route add default via 192.168.100.30
```

Where `192.168.100.30` is the address of the interface `eth0` of `r1`, connected to the lan. The router will be responsible of redirecting packets depending on his rules.

For `pc3`, we can add inside the startup file the following command:

```
route add default gw 192.168.100.30
```

To add a route that is not a default one, we can use the following command:

```
ip route add 192.168.100.29/24 via 192.168.100.25 # example
```

Have particular care in configuring routes. When asked, each device should reach any other device. This can be done by setting up routes in all routers. Remember that a packet should be able to go back as well (when dealing with IP).

So, for example, if we have a route to go to direction 1 to 2, we'll have to have also from 2 to 1.

Configuring DNS nameservers

We've seen how, with `pc1`, we've defined the `dns-nameserver`, this allows to have connectivity with the rest of the internet.

If we want to implement this for each machine, this will have to be enforced by adding an option inside the `/etc/resolv.conf` file. This can be done inside startup files, in this way:

```
echo 'nameserver 8.8.8.8' > /etc/resolv.conf
```

Testing connectivity

Once the lab has been started, we can test connectivity between devices using the `ping` command, giving as an argument the IP of the device we're trying to connect to.

To check if a device has the IP assigned correctly, we can use the following command:

```
ifconfig
```

An example (from):

```
ping 192.168.100.26 -c 4
```

We're defining that we're sending, with `-c`, 4 pings to that address,

We can also test connectivity to the internet by trying the following command:

Configuring IP addresses dynamically (DHCP)

Alongside configuring address statically, which could take some time, we can also configure addresses dynamically.

To do this, we need to configure a device as a DHCP server, which will handle the distribution of IP addresses to other devices configured as DHCP clients.

As for the DHCP server, we need to install `udhcpd` inside the machine, done in the startup file:

```
apt install udhcpd
```

Then, we need to define a configuration file for the server. This file is given by the `/etc/udhcpd.conf` file. Recommendation: create the directory for the device, then create inside it the etc directory, and then the configuration file.

```
start 192.168.100.18
end 192.168.100.29

max_leases 12

opt dns 8.8.8.8
option subnet 255.255.255.240
opt router 192.168.100.30
option domain pnd.test
option lease 600
```

With `start` we define the beginning of the range of IP addresses the server can distribute, while with `end` we define the last address.

Others are options (*which I'm not sure that actually needs to be configured*).

Then, to start the daemon, we insert the following lines inside the startup file:

```
dkpg -i /var/cache/apt/archives*.deb
apt install -f udhcp
```

```
udhpc -f /etc/udhcpd.conf
```

To configure a machine as a client, we use the `dhclient` command.

```
dhclient eth0
```

Capturing Traffic

Packets flow continuously across the network, and to capture them, you can use network traffic dump tools such as `dumpcap`, `wireshark` / `tshark`, or `tcpdump`. All of these tools are based on the `pcap` library (or `winpcap` in Windows) and can visualize and save the captured data. Furthermore, tools like Wireshark and `tcpdump` are capable of analyzing and decoding the captured packets.

In order to save the traffic capturing externally from the Kathara machines, the Professor provided us with a script. It works by adding two virtual interfaces, `veth0` and `veth1`, connecting one to the virtual bridge, assigning an IP address to the other, and enabling them both. In this way, we can type the following:

```
sudo wireshark
```

and get access to the monitoring of all the machine interfaces, including virtual ones.

This is the `connect-lab.sh` script, for future usage:

```
if [ $# -eq 0 ]
then
    echo "ERROR: usage $0 IPADDR/NETMASK [NETWORK_NAME]"
    exit 0
fi
echo "setting up host veth"

sudo ip link del veth1 type veth
sudo ip link add veth1 type veth
sudo ip addr flush veth0
sudo ip addr add $1 dev veth0
if [ -z "$2" ]
then
    echo "Network name not provided: assume only one lan"
```

```

exists..."
    bridge="$(sudo docker network ls | grep -e 'kathara_.*' | grep
-o '^[a-z0-9]*' | head -n 1)"
else
    bridge="$(sudo docker network ls | grep -e
"kathara_${USER}.*_2" | grep -o '^[a-z0-9]* ')"
fi

echo
echo "attaching veth1 to lan $2, bridge: kt-${bridge}"

sudo ip link set veth1 master kt-${bridge}

sudo ip link set veth0 up
sudo ip link set veth1 up
ip addr show dev veth1
ip addr show dev veth0

```

Another way to capture traffic is by using the `tcpdump` command inside the machine.

Specifically, one could specify:

```
tcpdump -w /shared/cap1.pcap
```

The `/shared` directory is a particular kind of directory, which can be accessed from outside the laboratory as well. It can be created, alongside other files, in the directory folder.

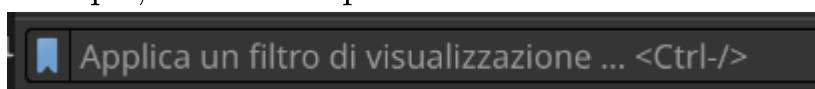
Analyzing traffic

Once we've captured traffic through `tcpdump`, or setup the virtual interface with the script provided above, we can analyze the packets that have been transmitted.

One major concern we may have when first taking a look at the screen is the amount of packages that have been tracked.

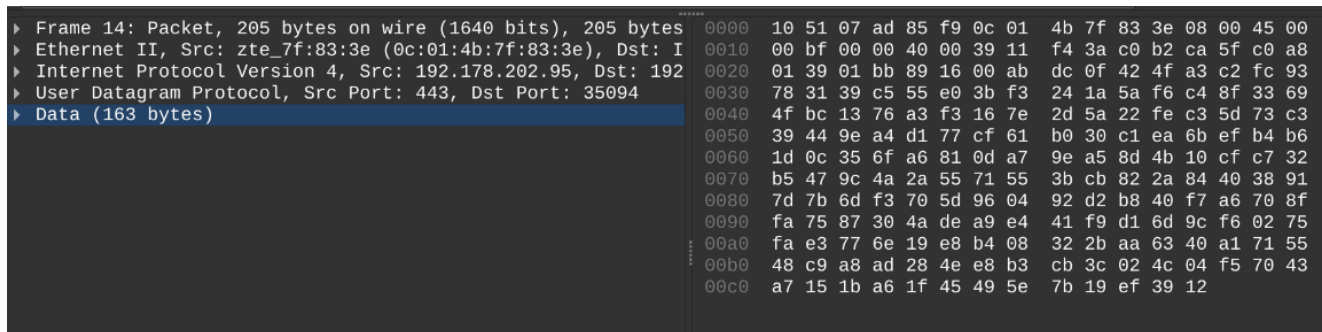
Luckily, Wireshark allows to apply filters to redirect our attention only to desired packets. This can be done essentially in two ways:

- Visual filters: applied in the search bar (photo below). Here we can specify, for example, the kind of protocol we want to focus on.



- Capture filters: they effectively reduce the amount of packets traced by only saving the content of the desired ones. To apply this kind of filters, we can use Berkeley Packet Filter (BPF) syntax. It's basically structured in this way: `protocol direction type`. For the `protocol` field we may use `tcp`, `udp`, `ip`, while for the `direction` either `src` or `dst`, and for `type` we could use `host`, `net`. Also, filters can be concatenated by using logical operands, like `&&`, `||` and `!`.

By clicking once on a packet, we get access to this:



The screenshot shows the Wireshark interface. The left pane (Packet List) shows a list of captured packets. The middle pane (Packet Details) shows the hierarchical structure of the selected packet (Frame 14). The right pane (Packet Bytes) shows the raw data of the packet in hexadecimal and ASCII.

```

> Frame 14: Packet, 205 bytes on wire (1640 bits), 205 bytes captured (1640 bits) on interface 0
> Ethernet II, Src: zte_7f:83:3e (0c:01:4b:7f:83:3e), Dst: Intel_Ethernet_Adapter (08:00:27:00:00:00)
> Internet Protocol Version 4, Src: 192.178.202.95, Dst: 192.168.1.1
> User Datagram Protocol, Src Port: 443, Dst Port: 35094
> Data (163 bytes)

```

The packet details pane shows the following structure:

```

Ethernet II, Src: zte_7f:83:3e (0c:01:4b:7f:83:3e), Dst: Intel_Ethernet_Adapter (08:00:27:00:00:00), Protocol: Internet Protocol Version 4, Length: 147
Internet Protocol Version 4, Src: 192.178.202.95, Dst: 192.168.1.1, Length: 60, TOS: 0, Flags: 0x00, Window: 0, Checksum: 0x0000, Options: []
User Datagram Protocol, Src Port: 443, Dst Port: 35094, Length: 60
Data (163 bytes)

```

The packet bytes pane shows the raw data in hexadecimal and ASCII:

```

0000  10 51 07 ad 85 f9 0c 01 4b 7f 83 3e 08 00 45 00
0010  00 bf 00 00 40 00 39 11 f4 3a c0 b2 ca 5f c0 a8
0020  01 39 01 bb 89 16 00 ab dc 0f 42 4f a3 c2 fc 93
0030  78 31 39 c5 55 e0 3b f3 24 1a 5a f6 c4 8f 33 69
0040  4f bc 13 76 a3 f3 16 7e 2d 5a 22 fe c3 5d 73 c3
0050  39 44 9e a4 d1 77 cf 61 b0 30 c1 ea 6b ef b4 b6
0060  1d 0c 35 6f a6 81 0d a7 9e a5 8d 4b 10 cf c7 32
0070  b5 47 9c 4a 2a 55 71 55 3b cb 82 2a 84 40 38 91
0080  7d 7b 6d f3 70 5d 96 04 92 d2 b8 40 f7 a6 70 8f
0090  fa 75 87 30 4a de a9 e4 41 f9 d1 6d 9c f6 02 75
00a0  fa e3 77 6e 19 e8 b4 08 32 2b aa 63 40 a1 71 55
00b0  48 c9 a8 ad 28 4e e8 b3 cb 3c 02 4c 04 f5 70 43
00c0  a7 15 1b a6 1f 45 49 5e 7b 19 ef 39 12

```

On the left, the different protocols used for the packet are specified, with the relative encapsulation. We can see how, for example, at transportation level the `udp` protocol is used, while at network level we have `IP`. If we click on the arrows, we can even get access to more information about that specific header of the network stack.

On the right, we have the encoding of the packet using hex values.

Also, by clicking on a packet with a right click, we get access to more options, like the "follow", which effectively allows us to see the flow of packets related to that one.

IPv6

IPv6, in contrast to IPv4, does not simply use one single address for a NIC, rather we can assign multiple addresses on a device.

Specifically, we can distinguish between a Link-Local address, assigned automatically to the machine, having a value typically starting with `fe80::`, used to communicate with devices sharing the network link, and the Global Unicast Address (GUA), used to obtain global connectivity and routing.

Configuring IPv6 addresses

The ways we can follow to configure an IPv6 address statically are pretty much equivalent to the ones seen for IPv4, with some slight modifications.

First thing first, the address we assign for the gateway router must **not** be the GUA of the gateway, rather its Link-Local address.

Now, to assign addresses we can do this in the following ways:

1. Assign the address through the `interfaces` file.

The syntax is similar to IPv4, however we should use `inet6` instead of `inet`.

```
auto eth0
iface eth0 inet6 static
    address 2001:db8:1::10/64
    gateway fe80::1
```

2. Through `ip` command.

```
ip -6 addr add 2001:db8:1::11/64 dev eth0
# for default route
ip -6 route add default via fe80::1 dev eth0
```

Notice how we've specified the interface also when adding the route, this has to be done as it's necessary to avoid ambiguity through different interfaces connected to different networks, as multiple networks might have the exact same Link-Local address, and so the device must know through which port they must redirect traffic to.

This is also true for the `ping6` command. In fact, when trying to ping a Link-Local address, we'll have to specify the interface through which we want to perform this action.

3. Through `ifconfig` and `route` commands.

Inside `dev.startup` file, insert the following:

```
ifconfig eth0 inet6 add 2001:db8:2::12/64
# gateway
route -A inet6 add default gw fe80::1 eth0
```

One extra note: to allow routers to forward packets, we must insert the following command in the startup file of the router:

```
sysctl -w net.ipv6.conf.all.forwarding=1
```

To add a routing to a specific subnetwork through next hop IP, we use the following:

```
ip -6 route add 2001:db8:1::/64 via fe80::2 dev eth0
```

Dynamic addressing in IPv6

By default, in Unix, addresses are assigned dynamically through the use of the Stateless Address Autoconfiguration (SLAAC) protocol. To manipulate how these addresses are generated, one must use the `sysctl` command, interacting with the Kernel.

One important notice: by default, Kathara does not allow the use of `sysctl` commands, and we need to open the lab in privileged mode. This can be done either by typing `kathara lstart --privileged`, or by using a shell file (like `lstart.sh`, provided in some labs). Notice that the former won't open the terminals of the labs automatically, so you'll have to open them yourself by typing: `kathara connect [dev_name]`.

One of the first parameters we can modify through `sysctl` is the `add_gen_mode`. This basically is used to tell how the IP will be generated. The possible values are:

- `0`, which allows to generate the address basing on EUI64 (default behaviour).
- `1`, does not generate a Link-Local, use EUI64 only for addresses generated by the autoconfiguration.
- `2`, generate stable private addresses, using the parameter given by `stable_secret`.
- `3`, generate stable private addresses using a random stable secret, if not set.

Here is an example inside a startup file:

```
sysctl -w net.ipv6.conf.eth0.addr_gen_mode=2
# we also need to add the stable secret for mode 2
sysctl -w net.ipv6.conf.eth0.stable_secret="fe80::1" # it's an
example
```

Another parameter is given by `use_tempaddr`. It is used to set the preference for Privacy Extensions, which have been introduced due to the fact standard SLAAC IP addresses were generated using the MAC address of the NIC, revealing sensitive information. Here are the possible values:

- `<=0`, disables Privacy Extensions.
- `=1`, enables Privacy Extensions, but the system will use public addresses rather than using temporary addresses.
- `>1`, enables Privacy Extensions and the system prefers using temporary addresses rather than public ones.

Example to maximise privacy:

```
sysctl -w net.ipv6.conf.eth0.use_tempaddr=2
sysctl -w net.ipv6.conf.eth0.temp_valid_lft=60 # 60 sec validity
sysctl -w net.ipv6.conf.eth0.temp_preferred_lft=30 # pref. time
```

Some extra parameters for temporary IP addresses can be defined with `temp_valid_lft` and `temp_preferred_lft`.

For additional info on `sysctl` commands, check out:

<https://www.kernel.org/doc/Documentation/networking/ip-sysctl.txt>.

Router Advertisement and Stateless DHCPv6

To make it possible for SLAAC to work, the router must send some special kind of message, the Router Advertisement, to advertise the network.

While, in a mixed setup given by SLAAC and Stateless DHCPv6, the router will handle the distribution of the IP prefix, and the DHCPv6 will distribute additional information like DNS servers.

To allow the router to send Router Advertisements, we should use the daemon `radvd`.

We'll have to create the `/etc/radvd.conf` file (which can be done, as we should remember, before starting the lab, simply by creating the `[dev]` folder, and putting it inside), and then insert something like this:

```
interface eth0 {
    AdvSendAvert on;
```

```
AdvOtherConfigFlag on; # stateless directive

prefix 2001:db8:1::/64 {
    AdvOnLink on;
    AdvAutonomous on; # SLAAC for IP
};

};
```

To configure a router as a DNS server, we'll use instead the `dnsmasq` daemon. We'll have to create the `/etc/dnsmasq.conf` file.

```
interface=eth0 # configure correct interface

dhcp-option=option6:dns-server,[2001:db8:1::1] # sends the IP of
the
# DNS server, typically the router itself

dhcp-range=::,constructor:eth0,ra-stateless # configure range,
# which is null in Stateless configuration, as DHCPv6 server only
# provides options
```

To start both daemons, we can do that in the startup file or in the labs directly. Here are the commands:

```
radvd -C /etc/radvd.conf
dnsmasq
```

We'll also have to configure clients to accept Router Advertisements message, and act as DHCPv6 clients.

```
sysctl -w net.ipv6.conf.eth0.accept_ra=1

# start DHCPv6 client in background
dhclient -6 -S eth0 # S flag used to indicate stateless
```

We can force the sending of a Router Solicitation message (for interface 0) through `rdisc6 eth0`.

DHCPv6 through Dibbler

We can either use the already seen `dhclient`, but also `dibbler`.

We'll need to install it through our startup files first:

```
dpkg -i /shared/dibbler-client*.deb
```

Then, we'll have to create the configuration file `/etc/dibbler/client.conf`:

```
log-level 7
iface eth0 {
    stateless
}
```

Then start the daemon through:

```
dibbler-client start
```

DHCPv6 Prefix Delegation

While dealing with Prefix Delegation, our DHCP router will function as a client for a ISP, by which he will receive a bunch of prefixes he will be able to distribute.

To configure this, we'll use `dibbler`, allowing to work with prefix delegation:

```
log-level 8
iface eth0 {
    pd
}
```

MTU Management

We can decide the maximum transfer dimension for packets within a certain link. This can be done with the following command:

```
ip link set dev eth0 mtu 1280 #example
```

It will need to be applied at all ends of a specific network link, in order to be operational.

We can test out the receive of a "Packet Too Big" message by forcing the MTU through the `ping` command, in this way:

```
ping -6 -s 1500 -M do 2001:db8:cafe:4::2
# with -M do we force the packet not to be fragmented
```

Additionally, we can trace the MTU among the different links we pass through by using the `tracepath` command:

```
tracepath -6 2001:db8:cafe:4::2
```

Stateful DHCPv6

When dealing with a Stateful DHCPv6, the server itself handles IPv6 addresses, instead of using SLAAC and having devices configure the IP themselves.

To make this work, we'll have to configure a device as a DHCPv6 server, and we'll use `dibbler`.

First, we install the server instance on the selected device:

```
apt install dibbler-server
```

Then, we configure it in `/etc/dibbler/server.conf` of the device:

```
log-level 7

iface eth0 {
    # timers for address renewal
    t1 1000
    t2 2000
    preferred-lifetime 3600
    valid-lifetime 7200

    # pool of addresses to distribute
    class {
        pool 2001:db8:1::100 - 2001:db8:1::200
    }

    # additional options to hand out
    option dns-server 2001:db8:1::1 # to hand out dns-server IP
}
```

We can then start the daemon in this way:

```
dibbler-server start
```

IMPORTANT NOTE: unlike in IPv4, IPv6 does not natively handle the IP address of the gateway server. We'll need to configure a router advertisement through `radvd` to have the router "broadcast" on the link local its address, as well as telling devices to get their address from DHCPv6 server, instead of using SLAAC.

Thus, we modify the `/etc/radvd.conf` file in this way:

```
interface eth0 {
    AdvSendAdvert on;
    AdvManagedFlag on; # tells client to use Stateful DHCPv6

    prefix 2001:db8:1::/64 {
        AdvOnLink on;
        AdvAutonomous off; # disable SLAAC
    };
};
```

Configuring the Firewall Rules

To configure necessary firewall rules, we can use the `iptables` command on the router (if we want a perimeter firewall) or on the device itself (if we want a local firewall).

`iptables` is based on the use of different tables, specifically, for now we operate with the `FILTER` table. Each table has multiple chains of rules. Specifically, we have `INSERT`, `OUTPUT` and `FORWARD`. The order in which we put rules in a chain is important, in fact, when a packet goes through the rule, it will be elaborated depending on the first matching rule.

To see configured rules for chains, we can use the following command:

```
iptables -L -n -v
```

Now, the `INPUT` chain is used specifically for packets going to the device itself (and not packets the device has to route). The `OUTPUT` chain is used, instead, for packets going outside the device. Finally, the `FORWARD` chain is used for packets that we have to forward, and typically we set these rules on a firewall.

There are tons of options we can use, specifically:

- `-A` to append (at the end of the chain) a rule to a certain chain. Example: `-A FORWARD`
- `-I` to insert at a specific position a rule in the chain. Example: `-I FORWARD 1`
- `-t` to specify the target table. By default, it's set to `FILTERING`.
- `-F` to flush the tables (deleting everything).
- `-P` to specify a specific policy. Example: `iptables -P FORWARD DROP`, drops by default all packets.
- `-p tcp --[s/d]port` allows to specify source port (`-p tcp --sport`) or destination port (`-p tcp --dport`).
- `-[s/d] [address]` to specify source or destination addresses.
- `-i` to specify ingress interface. Example: `-i eth0`
- `-o` to specify egress interface.

We can also decide to make the firewall stateful by adding modules. Modules can be added with `-m` option, and we can have the following options (provided after writing `-m state --state`):

- `NEW` : packet starting a connection.
- `ESTABLISHED` : packet part of an established connection.
- `RELATED` : packet related to another connection.
- `INVALID` : packet could not be identified.

Here are some examples of commands:

```
iptables -A FORWARD -s 0/0 -i eth0 -d 192.168.1.58 -o eth1 -p TCP
--sport 1024:65535 -m multiport --dport 83,443 -j ACCEPT
```

Note: `0/0` is used to specify every address.

```
iptables -A FORWARD -j REJECT # this should be added at the end
of all rules for Internet "Etiquette"
```

Important! Don't forget to add the rule to allow traffic to go back!! So, change the source and destinations if provided, and allow answers.

```
iptables -A FORWARD -d 0/0 -o eth0 -s 192.168.1.58 -i eth1 -p TCP
-m state --state ESTABLISHED -j ACCEPT
```

S-NAT Configuration (NAT)

NATting mechanism allows to connect devices using a private IP with devices using public IPs, thanks of the use of a firewall placed at the edge of the private network, performing Network Address Translation. Rules can be defined thanks to the `iptables` command, specifying the `nat` table, so `-t nat` .

We can distinguish between SNAT, used for connection requests coming from inside the private network, and DNAT, used for connections coming from the outside. Depending on the one we're using, we can use either the `POSTROUTING` or the `PREROUTING` chain.

The term `MASQUERADE` is used to determine the IP (which could change) of the interface we're having the packet leave.

So, to allow traffic from an internal network to reach an external network, and also to receive an answer, we have to do this:

```
iptables -t nat -A POSTROUTING -o eth1 -j MASQUERADE
```

`POSTROUTING` chain are rules processed on packets after the routing function has been performed, while `PREROUTING` are rules applied before the routing has been performed.

D-NAT Configuration

TBD

Simple IP Tunneling

Before diving into encryption, it is important to understand the concept of a tunnel. The universal `tun / tap` driver creates a virtual interface that encapsulates network traffic . The `tun` interface encapsulates the IP layer, while the `tap` interface encapsulates the Ethernet layer.

In a standard network setup, if a packet needs to cross the internet to a private network, it cannot be routed directly using its private IP. Tunneling solves this by taking the original IP packet (the payload) and encapsulating it inside a new IP header with public, routable IP addresses . To the applications running on the machine, the `tun0` interface acts exactly like a physical interface, seamlessly handling this encapsulation.

To create a basic tunnel between a local machine and a remote machine:

Step 1: Check existing interfaces

```
ip link
```

Step 2: Create a new tun virtual interface Create the interface by defining the remote and local IP addresses:

```
ip tunnel add tun0 mode ipip remote <ipaddressR> local  
<ipaddressL>
```

Step 3: Assign an IP address and activate the interface Give the tunnel an IP address (ensuring the remote machine uses a different IP, such as 10.0.0.2/30) and bring it up :

```
ip addr add 10.0.0.1/30 dev tun0  
ip link set tun0 up
```

IPsec Configuration

IPsec is a network layer protocol suite that provides security over IP. It operates primarily through two protocols:

- **Authentication Header (AH):** Provides data origin authentication, connectionless integrity, and replay protection. It does *not* encrypt the data .
- **Encapsulating Security Payload (ESP):** Provides all the benefits of AH, but additionally encrypts the payload to provide confidentiality .

IPsec can be used in two modes:

- **Transport Mode:** Protects the payload of the IP packet (typically used for Host-to-Host communication) .
- **Tunnel Mode:** Encapsulates and protects the entire inner IP packet by adding a new outer IP header (typically used for Gateway-to-Gateway VPNs)

IPsec behavior is governed by the **Security Policy Database (SPD)**, which determines which security services should be provided to IP packets, and the **Security Association Database (SAD)**, which stores the specific algorithms and keys used for the connection.

A. Manual Setup (Using **ipsec-tools**)

Note: Setting up Security Associations (SAs) manually is error-prone and generally used for educational purposes .

1. Security Policies (SPD) We use the `setkey` tool to define which security services should be applied to specific IP packets. The format is `protocol/mode/src-dst/level` .

For an IPv6 connection using ESP in Transport mode between `fec0::1` (Host A) and `fec0::2` (Host B), Host A's configuration looks like this :

```
spdadd fec0::2 fec0::1 any -P in ipsec esp/transport//require;  
spdadd fec0::1 fec0::2 any -P out ipsec esp/transport//require;
```

2. Security Associations (SAD) Next, manually add the encryption algorithm and key for the SA :

```
# Add an AH SA  
add fec0::1 fec0::2 ah 700 -A hmac-md5  
0xbf9a081e7ebdd4fa824c822ed94f5226;  
  
# Add an ESP SA  
add fec0::1 fec0::2 esp 701 -E 3des-cbc  
0xdafb418410b2ca6a2ba144561fab354640080e5b7ac0c133;  
add fec0::2 fec0::1 esp 801 -E 3des-cbc  
0xdafb418410b2ca6a2ba144561fab354640080e5b7ac0c133;
```

B. Automated Setup (Using `strongSwan`)

To avoid manual errors, it is better to rely on an IKE daemon like `strongSwan` to automatically establish SAs .

You manage `strongSwan` using the following commands :

```
# Start, stop, or check the status of the IPsec service  
ipsec start|stop|restart|status  
  
# Manage configurations and initiate connections via swanctl  
swanctl --load-conns  
swanctl --list-sas  
swanctl --initiate --child <connection>
```

OpenVPN Configuration

OpenVPN is an open-source software used to realize encrypted tunnels. It is highly popular because it typically relies on a single UDP (or TCP) port, allowing it to easily traverse firewalls and NAT environments . It operates in either a Static Key or Dynamic Key (PKI) mode.

A. Static Key Configuration

In this mode, both endpoints share a symmetric key. It is easy to configure, but the key never changes, meaning there is no forward secrecy: if the static key is ever compromised, all previously captured traffic can be decrypted .

1. Generate and Exchange the Key Generate the shared secret on the first router (`r1`):

```
openvpn --genkey secret secret.key
```

Since the key must be exchanged over an insecure channel, encrypt it using OpenSSL:

```
openssl enc -aes-128-cbc -e -a -in secret.key -out secret.key.enc
```

Transfer the encrypted key. Prepare `r2` to receive it, then send it from `r1` :

```
# On r2 (Receiver)
nc -l -p 9000 > secret.key.enc

# On r1 (Sender)
nc <r2IPAddress> 9000 < secret.key.enc
```

Decrypt the key on `r2` :

```
openssl enc -aes-128-cbc -d -a -in secret.key.enc -out secret.key
```

2. Configuration Files Create the configuration file `r1.conf` for the passive actor (the side waiting for connections) :

```
port 1194
proto udp
dev tun
secret secret.key
```

```
cipher AES-256-CBC
ifconfig 10.10.10.1 10.10.10.2
```

For `r2.conf`, point it to the remote server and flip the `ifconfig` IPs :

```
remote <r1IPAddress>
port 1194
proto udp
dev tun
secret secret.key
cipher AES-256-CBC
ifconfig 10.10.10.2 10.10.10.1
```

3. Run OpenVPN and Route Traffic Start OpenVPN on both ends :

```
# On r1
openvpn --config r1.conf

# On r2
openvpn --config r2.conf
```

To give visibility of another subnet through the tunnel, use the `route` option :

```
route <network> <netmask> <host>
```

B. Dynamic Key Configuration (PKI)

Dynamic configuration uses SSL/TLS and certificates for authentication. It provides Forward Secrecy because HMAC and encryption keys are dynamically generated with OpenSSL .

1. Certificates You will need a Certification Authority (CA) to sign certificates (e.g., using `easy-rsa` scripts). The required files include :

- `client.crt` / `server.crt` (CA signed public keys)
- `client.key` / `server.key` (CA signed private keys)
- `dh.pem` (Diffie-Hellman key exchange parameters)
- `ta.key` (Optional TLS HMAC authentication)

2. Run OpenVPN Start OpenVPN using the sample configuration files provided for the client and server :

```
# On the Server (VPN Gateway)
openvpn --config server.conf

# On the Client
openvpn --config client.conf
```

Generating a Digital Certificate

The fundamental problem with simple public key cryptography is that a user has no way to verify whether a public key they received actually belongs to the intended entity . This opens the door for Man-in-the-Middle (MITM) attacks, where an attacker intercepts communication and substitutes their own public key.

To solve this, we rely on a **Public Key Infrastructure (PKI)** and a **Certificate Authority (CA)**. The CA is a trusted third party that verifies an entity's identity and issues a Digital Certificate (based on the X.509 standard) that binds that identity to their public key .

`openssl` is a vast toolkit helpful for managing cryptography. One of the options we have is to inspect certificates as if we were clients:

```
openssl s_client -showcerts -connect www.paypal.com:443
</dev/null
```

By running this command, we connect as a client to a server (e.g., PayPal) and issue a request for its certificate. This outputs the entire "Certificate Chain," showing how the server's certificate is trusted because it was signed by an Intermediate CA, which in turn is trusted by a Root CA .

If we save the certificate data into a `.pem` file, we can decode it using the following command:

```
openssl x509 -in paypal.pem -text -noout
```

Decoding the file reveals critical details formatted to the X.509 standard, including the serial number, the signature algorithms, validity intervals, the Issuer's identity (the CA), and the Subject's identity (the owner of the certificate) .

Acting as your own Certificate Authority (CA)

To generate certificates for your own VPNs or web servers, you can act as your own Root CA.

Step 1: Create the CA's Self-Signed Root Certificate First, generate a public/private key pair and an X.509 certificate for the CA. Because it is the root, it signs its own certificate :

```
openssl req -x509 -newkey rsa:4096 -sha256 -days 3650 -keyout  
modelCA_key.pem -out modelCA_cert.pem
```

This generates a private key (`modelCA_key.pem`) secured by a passphrase and a self-signed public certificate (`modelCA_cert.pem`) valid for 10 years. During generation, you will be prompted to fill out subject fields (Country, Organization, Common Name).

Note: For the OpenSSL `ca` utility to work properly in subsequent steps, you typically need to create a specific directory structure (e.g., `demoCA` , `certs` , `newcerts` , `index.txt` , and `serial`) to keep track of the certificates it issues.

Step 2: Generate a Server Key and Certificate Signing Request (CSR)

Now, act as the web server or VPN server that needs a certificate. Generate its private key :

```
openssl genrsa -aes128 -out server_key.pem 2048
```

Next, create a Certificate Signing Request (CSR). This file contains the server's public key and its identity details, which will be sent to the CA for verification and signing:

```
openssl req -new -key server_key.pem -out server.csr -sha256
```

Step 3: Sign the Certificate Switching back to the CA role, you verify the CSR's subject information. If approved, the CA issues the final certificate by signing it with the CA's private key :

```
openssl ca -in server.csr -out server_cert.pem -md sha256 -cert  
modelCA_cert.pem -keyfile modelCA_key.pem
```

Once this signature is applied, the certificate cannot be modified by attackers.

Deploying the Certificate in Apache

Now that you have a signed certificate, you can configure a server (like Apache) to use it for HTTPS. The configuration is typically found in `/etc/apache2/sites-available/default-ssl.conf` .

```
<VirtualHost *:443>
    ServerName website.com
    DocumentRoot /var/www/html
    DirectoryIndex index.html

    SSLEngine On
    # The server's signed public certificate
    SSLCertificateFile /etc/apache2/ssl/server_cert.pem
    # The server's private key
    SSLCertificateKeyFile /etc/apache2/ssl/server_key.pem
    # The CA's certificate (used to complete the chain of trust
for clients)
    SSLCertificateChainFile /etc/apache2/ssl/modelCA_cert.pem
</VirtualHost>
```

Note: The `SSLCertificateChainFile` directive ensures that the server sends out the intermediate/root CA certificates alongside its own, providing the client with the full chain needed to verify authenticity .